Blockchain Chain of Custody Project

Group 13 - Track 2

Ethereum smart contract (Solidity) + Remix IDE

Arizona State University

Course Number: CSE 469

Members:

Ryan Clark

Olivia Pratt

Kishan Venkatesan

Zayn Shah

Faisal Alyousefi

1. Project Overview

The purpose of a chain of custody system is to maintain a trustworthy record of evidence handling from the moment it is collected until the final resolution of a case. Traditionally, investigators use paper forms or centralized digital systems to document who handled the evidence, where it was stored, and what actions were performed on it. This ensures that the evidence remains valid, credible, and admissible in court. Any errors or unlogged actions may be grounds for having evidence thrown out or challenged by opposing parties. Because of this, chain of custody forms must be secure, permanent, and tamper-proof.

In this project, our group implemented a digital version of a chain of custody system using the Ethereum blockchain. Since we chose Track 2 of the assignment, our goal was not to build a blockchain manually like in Track 1, but to leverage an existing framework. In our case, Ethereum smart contracts. We mapped every required chain-of-custody operation into functions inside a Solidity smart contract named ChainOfCustody. This contract stores evidence items, enforces state transitions, maintains detailed histories, and ensures that no one can change any past actions once recorded.

Using Ethereum allowed us to take advantage of the blockchain's built-in immutability. When a transaction is executed, the resulting state is permanently written to the blockchain, and each transaction becomes part of an ordered chain of blocks. This fits naturally with the idea of a chain of custody: once an action is logged, it cannot be undone or altered, which mirrors the real-world expectation that evidence handling must be documented clearly and permanently.

Throughout this project, we learned how to transition from a traditional software-engineering approach to a blockchain-based model. Instead of worrying about file formats, hashing algorithms, or building a ledger from scratch, we focused on designing the smart contract logic and ensuring the system's rules

2. Functional Requirements

Even though Track 2 doesn't require implementing a command-line tool, we mirrored the operations defined in the Track 1 specification. The smart contract provides the functional equivalents of all the original commands.

Add Evidence

Creators can add new evidence items using addEvidence(caseId, itemId) or add multiple items at once using addEvidenceBatch(caseId, itemIds). Each item starts in the CHECKEDIN state and is permanently associated with the case. The system enforces that each itemId is unique and cannot be re-added later.

Checkout

Owners or admins can check out evidence if it is currently stored, meaning it must be in the CHECKEDIN state. This operation updates the item's state to CHECKEDOUT and logs the action with the time and the actor's address.

Checkin

Owners or admins can return evidence to storage. The item must be in the CHECKEDOUT state. This sets the item's state back to CHECKEDIN and logs the event.

Remove

Creators can remove evidence from the system but only when it is currently CHECKEDIN. The reason for removal can be:

- DISPOSED
- DESTROYED
- RELEASED (requires owner information)
- NONE (generic removal)

After removal, the item enters a terminal state and cannot undergo further actions.

Show Cases

The contract allows viewing all case IDs that have at least one associated item. This mimics the show cases functionality from Track 1.

Show Items

For a given case, users with the appropriate permissions can retrieve all itemId values associated with that case.

Show History

Every item maintains a chronological record of actions. The getHistory() function supports:

- Returning the entire history
- Limiting results to the most recent n entries
- Reversing the result order to show newest actions first

Summary

The summary(caseId) function counts how many items are in each state, including:

- Checked-in
- Checked-out
- Disposed
- Destroyed
- Released

This matches the required summary command from the spec.

Permissions

Track 1 uses passwords and environment variables.

Track 2 replaces this with Ethereum roles:

systemCreator → Can grant or revoke any role

CREATOR -> Can add/remove evidence

OWNER -> Can checkin/checkout and view case data

ADMIN -> Supervisory role with elevated access

This permission model fully satisfies the original “who can perform which action” requirement.

3. Non-Functional Requirements

Although the main focus is on functionality, the non-functional aspects of a chain-of-custody system are extremely important. Our blockchain-based system meets key non-functional needs in the following ways:

Security

Ethereum provides digital signatures automatically. Every action is tied to the caller’s address, meaning there is no way to forge an interaction or claim someone else performed an action.

Integrity

Because each change becomes a blockchain transaction, and blocks include hashes of previous blocks, any attempt to tamper with history would break the entire chain. This mirrors the strict evidentiary standards required in real investigations.

Reliability

Smart contracts execute deterministically. The same inputs always produce the same outputs, reducing ambiguity in how evidence-handling rules are applied.

Transparency

Authorized users can view complete histories, making it easy to audit evidence handling. This is especially important for legal and investigative settings.

Immutability

Once a transaction is confirmed, neither the caller nor the system administrators can delete or modify it. Even mistakes remain permanently recorded, which is consistent with real-world forensic documentation.

Performance

On Remix VM or private Ethereum networks, performance is fast. On public blockchains, gas fees and throughput would affect performance, but our use case is purely educational and testing-based.

Scalability

Our system scales across many items and cases thanks to mapping-based data structures. While Ethereum has practical limits, the design itself doesn’t impose artificial restrictions.

4. Design Decisions

Choosing a blockchain framework and designing the structure of our smart contract required several thoughtful decisions.

Choosing Ethereum

We selected Ethereum because:

- It is the most widely used smart contract platform.
- Remix IDE makes development fast and accessible.
- Solidity is well-documented.
- Ethereum automatically handles hashing, block structure, and consensus.
- The blockchain model matches the project's goal of immutability.

Mapping Passwords to Roles

Instead of passwords, we used Ethereum accounts and mappings:

- isCreator[address]
- isOwnerRole[address]
- isAdmin[address]

This keeps the access control simple, secure, and in line with how smart contracts normally handle permissions.

Data Structures

We designed the following structures:

Item – stores the case, creator, ID, and current state

HistoryEntry – stores action, timestamp, state after action, actor, reason, metadata

Mappings – organize relationships:

- items[itemId] -> Item
- history[itemId] -> HistoryEntry[]
- caseToItems[caseId] -> itemIds[]
- _allCases[] -> every case ever added

These structures are optimized for quick lookups and low gas usage.

Batch Adding

Because the assignment allows adding multiple items to the same case in one command, we created:

function addEvidenceBatch(bytes16 caseId, uint32[] calldata itemIds) This improves usability and mimics the CLI.

Error Handling

We used Solidity require() statements to:

- block illegal state transitions
- prevent removing non-checked-in items
- block double-adding items
- enforce that RELEASED items have owner information

Separation of Roles

We made systemCreator the only address that can change user roles. This prevents conflicts and simplifies governance.

5. Challenges Faced

Several parts of the project required significant effort and problem-solving.

Understanding Track 2 vs Track 1

Track 1 requires manual binary file creation, AES encryption, custom hashing, and implementing a chain structure. Track 2 skips that but introduces blockchain concepts. We needed to reinterpret many parts of the assignment into something that makes sense on Ethereum.

Representing UUIDs as bytes16

A standard UUID must be converted from:

c84e339e-5c0f-4f4d-84c5-bb79a3c1d2a2 into 0xc84e339e5c0f4f4d84c5bb79a3c1d2a2

This required formatting and understanding byte packing.

Managing Multiple Roles

Testing required switching between multiple accounts within Remix. It took time to verify that each role had the correct permissions and that access control matched the assignment's rules.

History Ordering

The history system needed support for:

- full history
- limited entries
- reverse order

We had to manually index the arrays to support all the different views.

State Transition Validation

Some transitions are illegal:

- checkin without checkout
- checkout after removal
- double removal

We used many require() statements to enforce the correct workflow.

Interpreting Removal Reasons

The assignment allows different reasons for removing evidence. We mapped these to an enum and handled special cases like RELEASED requiring owner information.

6. Discussion: Is Blockchain Appropriate for Chain of Custody?

Blockchain has several strong qualities that make it appealing for a chain-of-custody system: immutability, transparency, and tamper resistance. Every action becomes part of a permanent record, which is exactly what investigators need. However, there are practical challenges that make blockchain a questionable fit in real-world forensic labs.

Reasons Blockchain Might Not Be Ideal

- Sensitive data cannot be deleted or modified, which may conflict with privacy laws.
- Ethereum transactions cost money on real networks.
- Low throughput makes it unsuitable for high-volume environments.
- Errors are permanent, meaning accidental logs cannot be corrected.
- Blockchain is public, unless using a private chain or encryption.

How These Issues Can Be Addressed

- Use private, permissioned blockchains like Hyperledger Fabric.
- Store only hashes on-chain and keep evidence details off-chain.
- Use role-based permissions to restrict viewing capabilities.
- Implement zero-knowledge proofs to maintain privacy while preserving integrity.

In short, blockchain works well academically and for certain enterprise contexts, but real forensic labs would need a specialized environment before adopting it.

7. How to Run the System

This project was designed to run entirely using Remix IDE. No installations or dependencies are required beyond a browser.

- **Step 1-- Open Remix** Go to: https://remix.ethereum.org
- **Step 2 -- Create Contract File:** Create a file named ChainOfCustody.sol and paste the entire smart contract code.
- **Step 3 – Compile:** Use compiler version **0.8.20**.

- **Step 4 – Deploy:** Use the **Remix VM (London)** environment and deploy the contract.
- **Step 5 -- Assign Roles:** Use these functions:

  grantCreator(address)

  grantOwner(address)

  grantAdmin(address)
- **Step 6 -- Use System Functions**

Call: addEvidence. addEvidenceBatch, checkout, checkin, removeItem, listItems, listCases, getHistory, summary

8. How Transactions Are Stored in the Blockchain

Each action performed through the contract becomes a blockchain transaction. Ethereum automatically adds:

- the caller's address
- timestamp
- gas used
- the new state
- the block hash

Our contract stores each action in _history[itemId], where each element is a HistoryEntry that includes:

- action type
- resulting state
- timestamp
- actor
- removal reason
- metadata

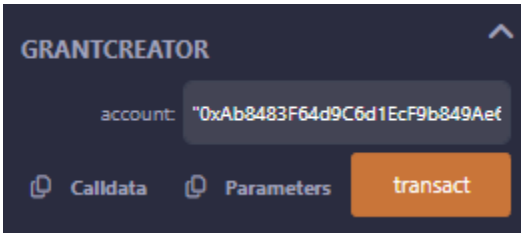
Because Ethereum stores data in a permanent ledger, once an entry is added, it can never be removed or edited.

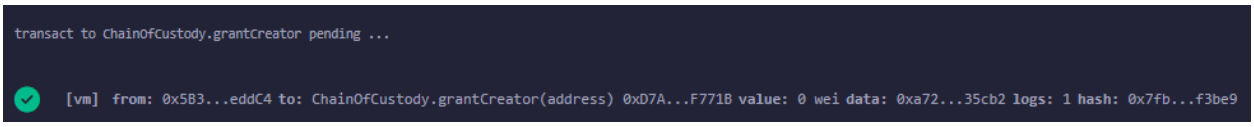
9. Screenshots Section

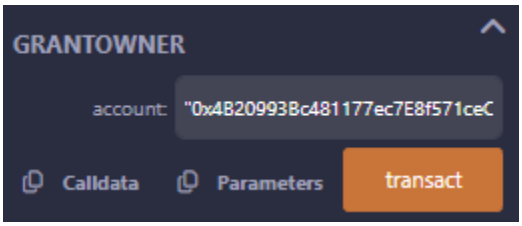
Figure 1 — Contract Deployment Screen

Description: Shows the ChainOfCustody contract deployed in Remix.

Figure 2 — Role Assignment Page







```
transact to ChainOfCustody.grantOwner pending ...

[vm] from: 0x5B3...eddC4 to: ChainOfCustody.grantOwner(address) 0xD7A...F771B value: 0 wei data: 0xdc9...c02db logs: 1 hash: 0xc5e...66848
```

(VERIFY OWNER IS TRUE, note bool: true)

```
isOwnerRole    0x4B20993Bc481177ec7E8f
0: bool: true
```

```
CALL [call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: ChainOfCustody.isOwnerRole(address) data: 0x1db...c02db

from            0x5B38Da6a701c568545dCfcB03FcB875f56beddC4
to              ChainOfCustody.isOwnerRole(address) 0xD7ACd2a9FD159E69Bb102A1ca21C9a3e3A5F771B
execution cost  2869 gas (Cost only applies when called by a contract)
input           0x1db...c02db
output          0x0000000000000000000000000000000000000000000000000000000000000001
decoded input   {
                    "address ": "0x4B20993Bc481177ec7E8f571ceCaE8A9e22C02db"
                }
decoded output  {
                    "0": "bool: true"
                }
```

Description: Granting CREATOR and OWNER roles using Remix buttons.

Figure 3 — Adding Evidence Items

```
ADDEVIDENCEBATCH
caseId:   0xc84e339e5c0f4f4d84c5bb79a3c1
itemIds:  [1004820154, 1004820155]
Calldata  Parameters  transact
```

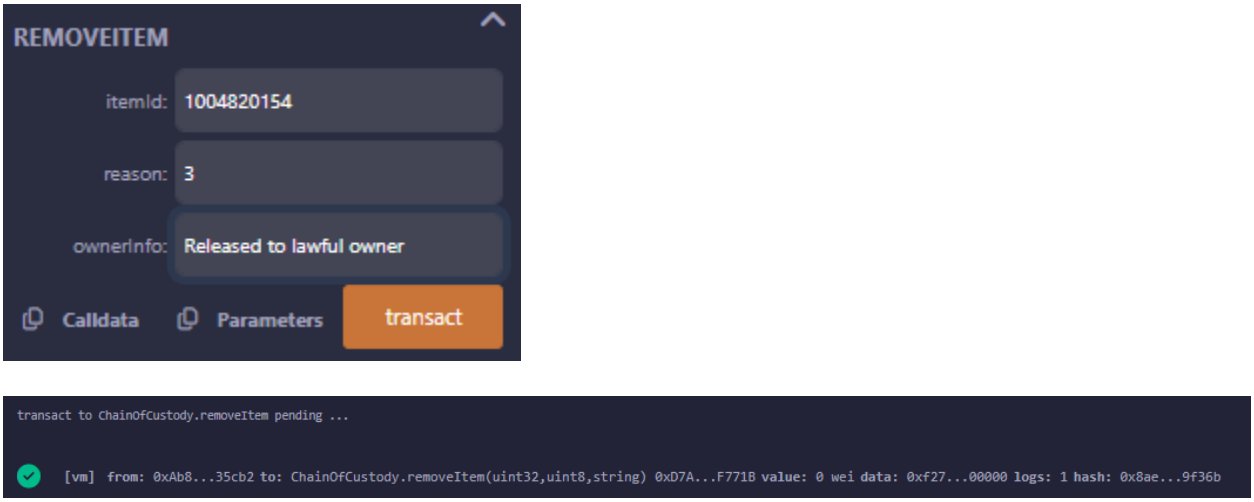
```
transact to ChainOfCustody.addEvidenceBatch pending ...

[vm] from: 0xAb8...35cb2 to: ChainOfCustody.addEvidenceBatch(bytes16,uint32[]) 0xD7A...F771B value: 0 wei data: 0xc27...456bb logs: 2 hash: 0xb87...ec264
```

Description: Example of addEvidenceBatch executed.

Figure 4 — Checking Out an Item

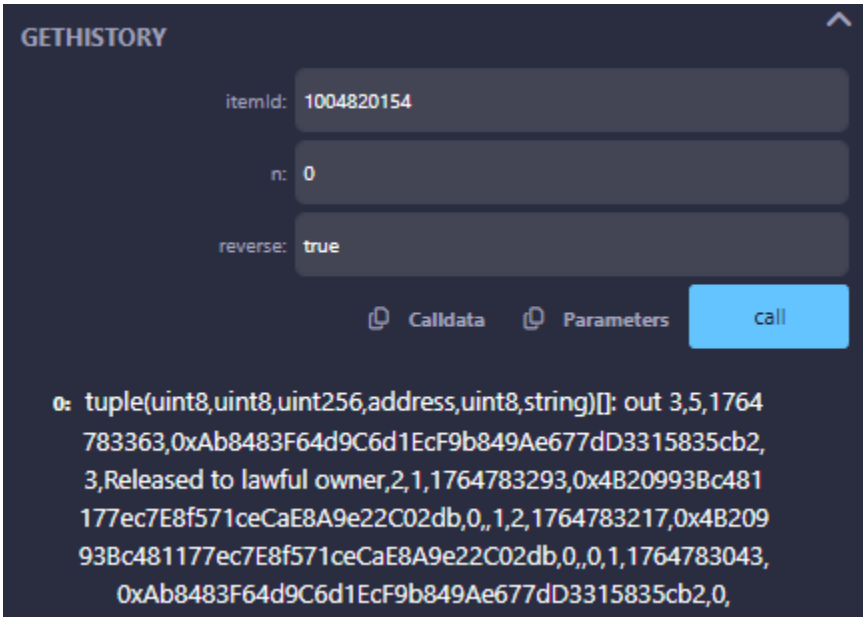
Description: Item 1004820154 moved to CHECKEDOUT state.

Figure 5 — Removing Evidence



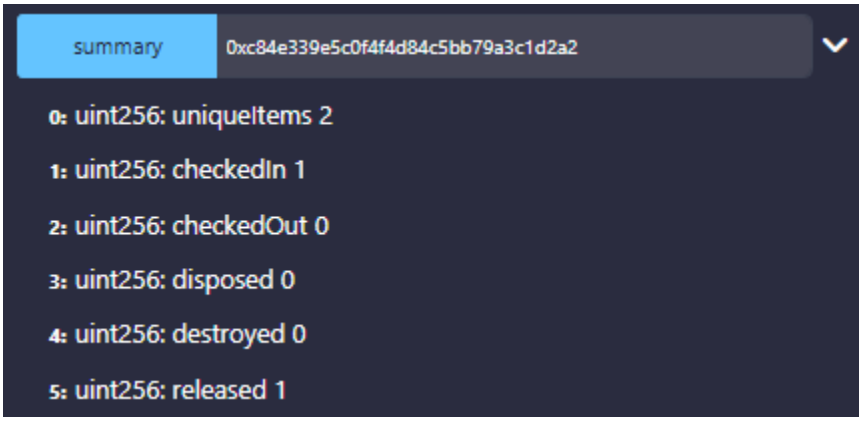
Description: Item removed with RELEASED reason.

Figure 6 — History Display



Description: History entries printed in order.

Figure 7 — Case Summary Output



```
call to ChainOfCustody.summary

CALL    [call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: ChainOfCustody.summary(bytes16) data: 0xf33...00000
```

Description: Summary() showing checkedIn, checkedOut, released, etc.